

TRD

BunkCalc

Smart Attendance Tracker for Indian College Students

Version 1.0 · May 2026 · Fayez

1. Tech Stack

Layer	Technology	Reason
UI Framework	React + Vite	Already used in DualCamera — zero ramp-up
Mobile Runtime	Capacitor	Same stack, proven on your test device (Moto Edge 50 Neo)
Styling	Tailwind CSS	Fast iteration, consistent design tokens
State Management	Zustand	Lightweight, already familiar from DualCamera
Local Storage	Capacitor Preferences API	Native key-value, persists across app restarts
Notifications	Capacitor Local Notifications	No backend needed, fully on-device
Share/Export	Capacitor Share + html2canvas	Share card PNG generation in-browser
Build	Android Studio + Capacitor CLI	APK for direct distribution + Play Store

No backend required for V1. All data is local. This is intentional — it eliminates auth complexity, server costs, and privacy concerns, while making offline use seamless.

2. Architecture

Data Model

All stored in Capacitor Preferences as JSON blobs:

Subject

```
{
  id: string // uuid
  name: string // "Data Structures"
  credits: number // 3 | 4
  threshold: number // 0.75 default
  schedule: {
    day: 0-6 // 0=Sun, 1=Mon ...
    slot: string // "09:00"
  }[]
  labMultiplier: 1|2 // labs count as 2
}
```

AttendanceRecord

```
{
  id: string
  subjectId: string
  date: string // ISO "2026-05-20"
  status: "present" | "absent" | "cancelled"
}
```

AppSettings

```
{
  semesterEndDate: string
  globalThreshold: number
  warningBuffer: number // default 0.05 above threshold
  notificationsEnabled: boolean
}
```

Bunk Budget Calculation

Core formula — runs reactively on every state change:

```
totalClasses = attended + absent
attendancePct = attended / totalClasses
safeBunks = floor(attended - threshold * (totalClasses + futureClasses))
where futureClasses = 0 for current state
```

The safe bunks number is the single most important computed value in the app. It must recalculate on every mark action with no visible latency.

3. Project Structure

```
src/
components/
  SubjectCard.tsx # bunk budget card per subject
  TodayList.tsx # daily marking list
  ShareCard.tsx # shareable PNG component
  AlertBanner.tsx # danger zone in-app banner
pages/
  Home.tsx # subject overview + today's list
  Setup.tsx # timetable onboarding wizard
  SubjectDetail.tsx # per-subject history + projection
  Settings.tsx # threshold, semester date, notifs
store/
  useSubjects.ts # Zustand slice: subjects CRUD
  useAttendance.ts # Zustand slice: mark/unmark
  useSettings.ts # Zustand slice: app config
lib/
  calculations.ts # bunk budget, projections
  notifications.ts # schedule / cancel local notifs
  storage.ts # Capacitor Preferences wrappers
  shareCard.ts # html2canvas export logic
App.tsx
android/ # Capacitor Android project
```

4. Key Implementation Details

Notification Scheduling

- On every attendance mark, recompute risk for all subjects
- Cancel existing scheduled notifications for that subject
- If new % < (threshold + warningBuffer): schedule a notification for next class day at 8AM

- Use Capacitor LocalNotifications.schedule() with a subject-scoped ID so they're replaceable
- Sunday 8PM: weekly summary notification listing at-risk subjects

Share Card Generation

- Render ShareCard component off-screen in a fixed-size div (375x667px)
- Use html2canvas to capture as PNG blob
- Call Capacitor Share.share() with the blob converted to base64 data URI
- Fallback: download PNG if share API unavailable

Cancelled Class Handling

- Cancelled class = neither present nor absent — does not affect attended OR total count
- Remove the class slot from denominator entirely
- Display as grey chip in history, not counted in any metric

Lab Session Handling

- If subject.labMultiplier = 2: each lab session mark counts as 2 classes in both numerator and denominator
- This matches how most Indian colleges count labs

Data Persistence Strategy

- Write to Capacitor Preferences on every state change (Zustand subscribe)
- Read all data on app cold start, hydrate Zustand store
- No async loading states on home screen — data already in memory after init

5. Build & Distribution

Debug APK (Beta)

```
npx cap sync android
npx cap open android
# In Android Studio: Build > Build APK
# Share via Telegram/WhatsApp
```

Release APK (Play Store)

```
# Generate signed release APK
./gradlew assembleRelease
# Sign with existing keystore (dualcamera-release.jks pattern)
# Upload to Play Store internal testing first
```

Performance Targets

Metric	Target
Cold start time	< 1.5s
Mark attendance action	< 100ms (local only)
Share card generation	< 2s
APK size	< 15MB
RAM usage	< 80MB

6. Technical Risks

Risk	Likelihood	Mitigation
html2canvas renders incorrectly on some Android WebViews	Medium	Test on 3+ devices early; have PNG fallback template
Capacitor Local Notifications blocked by Android battery optimization	High	Show one-time prompt to disable battery optimization on first notification setup
User data loss on app uninstall	Certain	V1 accepted; V2 add optional Google Drive backup
Timetable setup friction causing drop-off	Medium	Keep wizard to max 3 screens; allow skipping and editing later